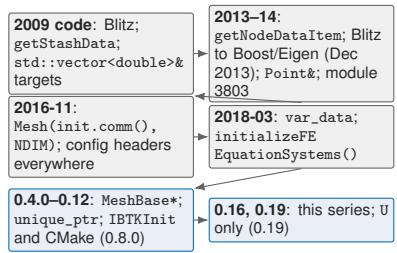
X3. OLD IBAMR AND LIBMESH CODE, MAPPED TO CURRENT CODE

How to read the course and lab programs of 2009–2024, what each old pattern was, and what it is in IBAMR 0.16 and 0.19 with CMake: a map for porting, labelled legacy



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part X, archive • Chapter X3

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

HOW TO USE THIS CHAPTER

- Legacy. Every code line marked “old” here is history, quoted with its era and file so that you can recognize it. Do not type it: the “new” line, from this series’ examples or from IBAMR 0.19.0’s headers, is what to write.
- First date the code (slides 3–4): the Makefile’s paths tell you which IBAMR it was written for.
- Then follow the map: one slide per kind of change, in the order a port meets them (build, compile, run, look at the output); slides 37–39 map every folder of `examples/legacy`, and four 2013–14 examples, to its maintained example (or to none).

> The short version is chapter 3’s porting checklist; this chapter is its long form, with the history of each change. “Changed in” gives an IBAMR version only where IBAMR’s changelog states it; otherwise the bracket between two course installs.

The code quoted comes from the 2014 and 2016-11 course folders and those kept in `examples/legacy`, the 2014 lab examples, the course decks and install notes (2012–2024) and IBAMR 0.19.0’s source.

DATING OLD CODE BY ITS MAKEFILE PATH (1/2): UNC, 2011–2020

Open the Makefile and read the two lines that point at IBAMR: `IBAMR_SRC_DIR`, the source, and `IBAMR_BUILD_DIR`, the compiled library. Their paths say where and when the program was built.

path in the Makefile	what it means
`$(HOME)/sfw2/...`	a source build in a home folder on UNC's KillDevil, 2012–13 (chapter X2)
`/usr/bin/sfw/...`	the lab's own Linux machine, built as root, 2011–13 (chapter X2)
`/nas02/apps/ibamr-3803/OFED-1.5.2/IBAMR2`	UNC's module `ibamr/3803` (Kure and KillDevil, about February 2014): probably SVN revision 3803, older than IBAMR's first tag, v0.1-rc1 (May 2014)
`/nas02/apps/ibamr-2016-11/...`	UNC's module `ibamr/2016-11` (November 2016): the course's 2016–17 ports
`/nas/longleaf/apps-dogwood/ibamr/2018-03/...`	UNC's Dogwood, module `ibamr/2018-03` (March 2018): a snapshot of IBAMR from before 0.3.0; most UNC folders of `examples/legacy`
`/nas/longleaf/apps-dogwood/ibamr/0.6.0/...`	Dogwood's module `ibamr/0.6.0` (2020): the UNC folders `7-*` and `9-*`

The Makefiles of `examples/legacy` and of the 2014 and 2016-11 course folders; IBAMR's release tags; the instructor's install notes; the code check of October 2026.

DATING OLD CODE BY ITS MAKEFILE PATH (2/2): ARIZONA, AND THIS SERIES

path in the Makefile	what it means
`/contrib/<group>/ibamr/0.7.1/...`	a University of Arizona build of IBAMR 0.7.1 (the UA rubber band; the AZ copies of the rubber band, moving cylinders and triangle in `originals/Old files`)
`/groups/<group>/ib10/...`	the UA autotools build of 2022–24, used by almost all UA folders; 0.10 judged from the folder name only
`/groups/<group>/ib16/...`	a UA autotools build of July 2025, presumably 0.16
no Makefile, a `CMakeLists.txt` with `find_package(IBAMR 0.16.0)`	this series' examples: IBAMR 0.16.0 and 0.19.0, built by autoibamr (chapter 2)

The UA rubber band's Makefile, for example:

```
IBAMR_SRC_DIR = /contrib/<group>/ibamr/0.7.1/include/IBAMR
IBAMR_BUILD_DIR = /contrib/<group>/ibamr/IBAMR-0.7.1
LIBSNEW = $(LIBS) -lnetcdf -lcurl
```

An autotools Makefile includes `$(IBAMR_BUILD_DIR)/config/make.inc` near its top; autoibamr installs no `make.inc`, and the next release, IBAMR 0.20 (in development in October 2026), removes autotools altogether. A Makefile path is therefore a date, not a recipe: today's examples build with CMake (slide 6).

`examples/legacy/UA-Examples` Makefiles and `originals/IBAMR Lectures/Files/Makefile` (`<group>` in place of the group's name); IBAMR's release tags; the code check of October 2026.

FIVE ERAS OF ONE EXAMPLE: THE RUBBER BAND

era	what the step changed in `main`
2014 (module 3803)	`PetscInitialize(&argc,&argv,PETSC_NULL,PETSC_NULL)`; the `*_prefix_config.h` includes commented out; `app_namespaces.h` in the middle of the includes; `.isNull()` tests
2016-11	`IBAMR_config.h`, `IBTK_config.h`, `LData.h` and `LDataManager.h` added (November 2016; the other 2016-11 mains mark them "added for MODULE 2016-11 NAB", N. Battista, and the headers "Added for IBAMR KD Module 2016-11 by NAB (1/27/17)"); `app_namespaces.h` moved after all other includes
2018-03 (Dogwood)	a fresh copy of IBAMR's `IB/explicit/ex1` main, adapted: `NULL`, a solver-type switch, named `Pointer<CartGridFunction>` objects, the `periodic_shift` loop
UA, 0.7.1 and `ib10` (2020–24)	the same main; an input whose lines end with `//` again, as in 2014 (a Windows line-ending workaround, slide 29)
0.16 / 0.19	`IBTKInit`; config headers gone; `IBTK::rel_equal_eps`, `IBTK_MPI`; `nullptr`; `snprintf`; `PetscInt`; a `CMakeLists.txt`

"I was only able to get a working IB example by taking more recent main, makefile, and input files and modifying them for my example" (the 2018 Dogwood porting slides): that is why the 2018 mains look like upstream's, and why the next slides follow the order of a port rather than the history.

Era-to-era diffs of the rubber band's `main` (2014, 2016-11, 2018-03, UA, IB/01); the instructor's 2018 porting slides.

BUILD: A MAKEFILE WITH `MAKE.INC`, THEN CMAKE

Old (the rubber band's Makefile, 2014)

```
IBAMR_SRC_DIR = /nas02/apps/ibamr-3803/OFED-1.5.2/IBAMR2
IBAMR_BUILD_DIR = /nas02/apps/ibamr-3803/ibamr-objs-opt
...
include $(IBAMR_BUILD_DIR)/config/make.inc
...
SOURCES = main.C update_target_point_positions.C ...
OBJS = main.o update_target_point_positions.o update_springs.o
main2d:
        if (test -f stamp-3d); then $(MAKE) clean; fi
        touch stamp-2d
        $(MAKE) PDIM=2 main-2d
```

New (IB/01's `CMakeLists.txt`)

```
cmake_minimum_required(VERSION 3.15...3.30)
project(ib01_rubber_band LANGUAGES C CXX)
find_package(IBAMR 0.16.0 REQUIRED)
add_executable(main2d main.cpp)
target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)
```

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 4
mpirun -np 4 ./build/main2d input2d
```

Extra sources go into `add_executable`; 3D uses `IBAMR::IBAMR3d` and `main3d`.

Today's error with an old Makefile: `Makefile:27: /contrib/<group>/ibamr/IBAMR-0.7.1/config/make.inc: No such file or directory` (chapter 3).
Changed in 0.8.0: "IBAMR can now be configured, compiled, and installed with CMake instead of with autotools".

The 2014 course Makefile; `examples/IB/01-rubber-band`; IBAMR's changelog 0.7.1 to 0.8.0; chapter 3.

MAKEFILE LEFTOVERS: LIBSNEW, -LNETCDF -LCURL, STAMPS, .C FILES

- **LIBSNEW.** The 2014 IBFE decks said “At the beginning of the Makefile declare LIBSNEW”, `LIBSNEW = $(LIBS) -lnetcdf -lcurl`, “Use this instead of the $(LIBS) flag”. The 2014 IBFE Makefiles link with `$(LIBSNEW)`, and so do five UNC and two UA IBFE Makefiles of 2018–24. Sixteen of the 21 UNC Makefiles write `LIBS = $(LIBS) -lnetcdf -lcurl` instead: a dead line, since the `make.inc` included after it sets `LIBS` again. Neither is needed today: netCDF has come in through libMesh since IBAMR 0.6.0, and a CMake build has no such line.
- **Stamps.** `stamp-2d` and `stamp-3d` made `make clean` the objects when you switched dimension; with CMake, `main2d` and `main3d` are separate targets.
- **File names.** `main.C` and `update_*.C` became `main.cpp` and `update_*.cpp` in this series (CMake accepts `.C`); IBAMR’s own examples are `example.cpp`.
- **One program instead of near-copies**: IBFE/01 (`MOTION`) and IBFE/06 (`FIBER_MODE`) replace three course folders each with one input key.

The 2014 IBFE Makefile slides; `examples/legacy` Makefiles; the October 2026 code check (C1); the maintained READMEs.

CONFIGURATION HEADERS THROUGH FOUR ERAS

- Up to 2013: `#include <IBAMR_prefix_config.h>`, `<IBTK_prefix_config.h>`. The December 2013 notes and the 2014 module slides say to comment them out; the 2014 code has them commented.
- 2014 IBFE and 2016–2024: `IBAMR_config.h`, `IBTK_config.h`, `SAMRAI_config.h`, added to every IB program and every `update_*.h` for module 2016-11.
- Now: delete the first two and keep `<SAMRAI_config.h>`; `ibamr/app_namespaces.h` already includes `<ibamr/config.h>`.

```
// 2016-11, update_beams.h
#include <IBAMR_config.h>    //Added for IBAMR KD Module
      2016-11 by NAB (1/27/17)
#include <IBTK_config.h>
#include <SAMRAI_config.h>
```

In IBAMR's autotools templates today: `#pragma message("This file has been deprecated: use ibamr/config.h instead")`.

Today's error (almost every course program stops here first; the three UA fiber-stress programs and the 2014 IB programs do not include `IBAMR_config.h` or `IBTK_config.h`): `main.cpp:31:10: fatal error: IBAMR_config.h: No such file or directory`. CMake installs have no `IBAMR_config.h` or `IBTK_config.h`; the version of the deprecation is not stated.

The instructor's December 2013 and March 2014 notes; the 2016-11 course code; IBAMR 0.19.0's `config/IBAMR_config.h.tmp.in`; chapter 3.

INCLUDES THAT BELONG WHERE

- 2014: `<ibamr/app_namespaces.h>` in the middle of `main`'s includes, and inside `update_*.h` headers.
- 2016-11: `#include <ibtk/LData.h> //added for MODULE 2016-11 NAB` before `<ibtk/LDataManager.h>`; `app_namespaces.h` moved after all other includes; each `update_*.h` got about twenty includes.
- The lab's 2013 notes had already added `#include <ibtk/LDataManager.h>` to `main.C` on the lab's machine.
- IBTK's rule, in `app_namespaces.h`: "This header file may be included in application codes, but it MUST NOT be included in any other header (.h) or inline (-inl.h) file in the library."

New (IB/07's `update_target_point_positions.h`)

```
#include <ibtk/LData.h>
#include <ibtk/LDataManager.h>
#include <PatchHierarchy.h>
...
void update_target_point_positions(
    SAMRAI::tbox::Pointer<SAMRAI::hier::PatchHierarchy<NDIM> >
      hierarchy,
    IBTK::LDataManager* l_data_manager, ...);
```

Fully qualified types, no `app_namespaces.h`. "IBAMR 0.16 needs LData.h for LDataManager.h to compile; 0.19 does not."

The 2014 and 2016-11 course code; the 2017 deck "How to update examples to new install"; `examples/IB/07-moving-plates`; IBAMR 0.19.0.

STARTING IBAMR: PETSCINITIALIZE AND SAMRAI, THEN IBTKINIT

Old (IB, 2018–2024; 2014 and 2016-11 with PETSC_NULL)

```
PetscInitialize(&argc, &argv, NULL, NULL);
SAMRAI_MPI::setCommunicator(PETSC_COMM_WORLD);
SAMRAI_MPI::setCallAbortInSerialInsteadOfExit();
SAMRAIManager::startup();
...
SAMRAIManager::shutdown();
PetscFinalize();
```

New (IB/01; IBFE/01 adds the second line)

```
IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
const LibMeshInit& init = ibtk_init.getLibMeshInit();
```

No shutdown calls: “IBTKInit also shuts them down when it goes out of scope.”

IBFE: `LibMeshInit init(argc, argv);` instead of `PetscInitialize`.

What `IBTKInit` does (0.19.0, `IBTKInit.cpp`): starts libMesh (or PETSc without it), calls `SAMRAIManager::setMaxNumberPatchDataEntries(2048)`, sets the SAMRAI and IBTK communicators, `setCallAbortInSerialInsteadOfExit`, `SAMRAIManager::startup()`; its destructor shuts down. Changed in 0.8.0: “IBAMR now requires the initialization object IBTKInit to be created upon program start.” The old start-up still compiles and runs (IBAMR’s own `IB/explicit/ex0` and `IBFE/explicit/ex0` still use it), but keeps SAMRAI’s default of 256 patch-data entries, a risk for runs with many variables (inferred, not observed).

`examples/legacy` mains; `examples/IB/01`, `IBFE/01`; IBAMR 0.19.0’s `ibtk/src/utilities/IBTKInit.cpp` and changelog 0.7.1 to 0.8.0; chapter 3.

PETSC_NULL, IBTK_MPI AND THE END-OF-LOOP TEST

Old

```
PetscInitialize(&argc,&argv,PETSC_NULL,PETSC_NULL); // 2014,
      2016-11
PetscInitialize(&argc, &argv, NULL, NULL);          // 2018-03
while (!MathUtilities<double>::equalEps(loop_time,
      loop_time_end)
       && time_integrator->stepsRemaining())
SAMRAI_MPI::sumReduction(&F_sum[0],NDIM);
```

New (IB/01)

```
while (!IBTK::rel_equal_eps(loop_time, loop_time_end)
       && time_integrator->stepsRemaining())
IBTK_MPI::sumReduction(&F_sum[0], NDIM);
```

- `PETSC_NULL` became `NULL` between the course's 2016-11 and 2018-03 code; PETSc itself deprecated `PETSC_NULL` in 3.19 (`PETSC_DEPRECATED_MACRO(3, 19, 0, "PETSC_NULLPTR", )`), so old code compiles with a warning. With `IBTKInit` there is no `PetscInitialize` in user code at all.
- Changed in 0.8.0: "All SAMRAI_MPI calls in the library have been replaced with respective IBTK_MPI calls." Optional in user code: `SAMRAI_MPI::sumReduction` still exists, and `equalEps` still compiles.

The course mains of 2014, 2016-11 and 2018-03; `examples/IB/01`; PETSc 3.23.3's `petscsys.h`; IBAMR changelog 0.7.1 to 0.8.0.

SMALL THINGS A FRESH UPSTREAM `main` BRINGS

old (2014, 2016-11)	2018-03 and now
`... && !app_initializer->getVisItDataWriter().isNull();`	`... && app_initializer->getVisItDataWriter();`
`registerVelocityInitialConditions(new muParserCartGridFunction("u_init", ...));`	a named `Pointer<CartGridFunction> u_init = new muParserCartGridFunction(...);`, then `registerVelocityInitialConditions(u_init);`
`new INSStaggeredHierarchyIntegrator(...)`	a `solver_type` switch (`STAGGERED` or `COLLOCATED`) read from `Main`
`getPeriodicShift().min() > 0` and `static_cast<RobinBcCoefStrategy<NDIM>*>(NULL)`	a `periodic_shift.min() > 0` loop setting `u_bc_coefs[d] = NULL`, now `nullptr`
`sprintf(temp_buf, "%05d", iteration_num);`	now only (the 2018-03 and UA mains still use `sprintf`): `std::snprintf(temp_buf, sizeof(temp_buf), "%05d", iteration_num);`
`int local_sz;`	now only: `PetscInt local_sz;`

None of these stops a build with autoibamr's PETSc (`int local_sz` would with a PETSc built for 64-bit indices); they come for free when you start from a current example's `main` and move your own parts into it, which is the fastest port.

Era-to-era diffs of the rubber band's `main`; `examples/IB/01-rubber-band/main.cpp`.

REACHING THE LAGRANGIAN DATA IN THE 2009 CODE: GETSTASHDATA

Old (Boyce Griffith's 2009 `moving_plates` example, shown in the course on 29 January 2013; its title slide says 2012)

```
const int lag_node_index_idx =
      lag_manager->getLNodeIndexPatchDescriptorIndex();
tbox::Pointer<hier::PatchLevel<NDIM> > level =
      hierarchy->getPatchLevel(finest_ln);
for (SAMRAI::hier::PatchLevel<NDIM>::Iterator p(level); p; p++)
{
    SAMRAI::tbox::Pointer<SAMRAI::hier::Patch<NDIM> > patch =
      level->getPatch(p());
    const SAMRAI::hier::Box<NDIM>& patch_box = patch->getBox();
    const SAMRAI::tbox::Pointer<LNodeIndexData> idx_data =
      patch->getPatchData(lag_node_index_idx);
    for (LNodeIndexData::LNodeIndexIterator it =
      idx_data->lnode_index_begin(patch_box);
         it != idx_data->lnode_index_end(); ++it)
    {
        const LNodeIndex& node_idx = *it;
        const int lag_idx = node_idx.getLagrangianIndex();
        SAMRAI::tbox::Pointer<IBTargetPointForceSpec> force_spec =
      node_idx.getStashData<IBTargetPointForceSpec>();
        if (!force_spec.isNull())
```

- The loop walks the patches of the finest level and, on each, the Lagrangian nodes inside the patch box; each node carries its "stash" of force specifications.
- The FAQ of the time put it in one line: `tbox::Pointer<T> spec = node_idx.getStashData<T>();`
- Note: the slides kept quoting this FAQ line until 2020, next to code that had long used the form of the next slide.

The January 2013 tutorial (dated 2012 on its title slide; its e-mail header lines withheld); the 2013–2020 decks on target points.

REACHING THE LAGRANGIAN DATA SINCE 2013: LMESH NODES AND GETNODEDATAITEM

Old (February 2013; the 2014 course code)

```
Pointer<LMesh> mesh = l_data_manager->getLMesh(finest_ln);
vector<LNode*> nodes;
nodes.insert(nodes.end(), mesh->getLocalNodes().begin(),
      mesh->getLocalNodes().end());
nodes.insert(nodes.end(), mesh->getGhostNodes().begin(),
      mesh->getGhostNodes().end());
...
for (vector<LNode*>::iterator it = nodes.begin(); it !=
      nodes.end(); ++it)
  {
    LNode* node_idx = *it;
    IBTargetPointForceSpec* force_spec =
      node_idx->getNodeDataItem<IBTargetPointForceSpec>();
    if (force_spec == NULL) continue;  // skip to next node
```

New (IB/07)

```
Pointer<LMesh> mesh = l_data_manager->getLMesh(finest_ln);
std::vector<LNode*> nodes = mesh->getLocalNodes();
nodes.insert(nodes.end(), mesh->getGhostNodes().begin(),
      mesh->getGhostNodes().end());
for (LNode* const node : nodes)
{
    IBTargetPointForceSpec* const force_spec =
      node->getNodeDataItem<IBTargetPointForceSpec>();
    if (force_spec == nullptr) continue; // skip to next node
```

Local and ghost nodes, a range-based loop, `nullptr`. In 0.19.0: `LNode.h` `T* getNodeDataItem() const;`, `LMesh.h` `getLocalNodes()`, `getGhostNodes()`.

Changed between the 2009 code (as the January 2013 deck still showed it) and the February 2013 decks (no IBAMR version stated). Chapter 9a explains the loop.

The February 2013 deck; the 2014 course code; `examples/IB/07-moving-plates`; IBAMR 0.19.0 headers.

TARGET POINTS: FOUR TYPES FOR ONE POSITION

era	the line that gets the target position
2009 code, January 2013 deck	`std::vector<double>& X_target = force_spec->getTargetPointPosition();`
February 2013 (Blitz era)	`TinyVector<double,NDIM>& X_target = force_spec->getTargetPointPosition();`
December 2013 – March 2014	"comment out the following … and add the last line": `//TinyVector<double,NDIM>& X_target = ...; //IBTK::Vector<double,NDIM>& X_target = ...; Point& X_target = force_spec->getTargetPointPosition();`
2014–2024 course code	the two old lines commented ("Depending on the version of IBAMR, you need to select one of the ways …"), `Point& X_target = ...` live
now (0.16, 0.19)	`Point& X_target = force_spec->getTargetPointPosition();`; in 0.19.0 `IBTK::Point& getTargetPointPosition();`, with `Point` and `Vector` both `VectorNd = Eigen::Matrix<double, NDIM, 1>`

Changed with the move from Blitz to Boost and Eigen in December 2013: the notes say that IBAMR's developers "now provide NDIM-dimensional Matrix, Vector, and Point classes that are based on Eigen … Vector and Point are interchangeable". Unchanged from 0.16 to 0.19.

The January and February 2013 decks; Boyce Griffith's December 2013 notes (in the instructor's install notes); the March 2014 module slides; IBAMR 0.19.0's `IBTargetPointForceSpec.h` and `ibtk_utilities.h`.

INDEX RANGES: GETLAGRANGIANSTRUCTUREINDEXRANGE AND [FIRST, SECOND)

Old (2014; the same in 2009 with `lag_manager`)

```
const std::pair<int,int>& plate2d_left_lag_idxs =
    l_data_manager->getLagrangianStructureIndexRange(0,
      finest_ln);
```

New (IB/07)

```
const std::pair<int, int> plate2d_left_lag_idxs =
    l_data_manager->getLagrangianStructureIndexRange(0,
      finest_ln);
```

- In 0.19.0 the function returns the pair by value; the old `const&` still compiles (the temporary's lifetime is extended), so this is style, not a fix.
- "Each range is `[first, second)`: `second` is one past the last index of that plate" (IB/07's comment): test `first <= lag_idx && lag_idx < second`.
- The indices are global: the structures of `structure_names` numbered in order (chapter 9a).

The 2014 course code; `examples/IB/07-moving-plates`; IBAMR 0.19.0's `LDataManager.h`.

SPRINGS (1/2): FROM GETRESTINGLENGTHS() TO GETPARAMETERS()

era	code	effect
2012 FAQ	`tbox::Pointer<IBSpringForceSpec> spring_spec = node_idx.getStashData<IBSpringForceSpec>();`	–
February 2013	`std::vector<double>& resting_length = spring_spec->getRestingLengths(); resting_length[0]+=0.01*dt;`	works (a reference)
"newer IBAMR"	Boyce Griffith's answer: springs have any number of parameters, the first the spring constant, the second the rest length: `spec->getParameters()[spring_number][1]`	–
2014 course code and slides	`double resting_length = spring_spec->getParameters()[0][1]; ... resting_length+=0.01*dt;`	no effect: a local copy
2018 course code	`std::vector<double>& spring = spring_spec->getParameters()[0]; ... spring[1] += 0.01*dt;` (marked `//Change 2-14 <name>`)	works (first spring only)
now (IB/06)	a loop over `spring_spec->getParameters()`, next slide	works (every spring of the point)

The March 2014 module slides: "At this time no changes appear to be needed to access either the resting lengths or the stiffnesses." Changed: `getRestingLengths()`, `getStiffnesses()` to `getParameters()` between the February 2013 deck and October 2013 (the curvature example's `update_springs.C` of 9 October 2013 already uses `getParameters()`; not versioned).

The 2012–2020 decks on springs; the lab's install notes (Boyce Griffith's answer); the 2014 and 2018 course code; `examples/IB/06-growing-springs`.

SPRINGS (2/2): THE COPY THAT DID NOTHING, AND THE FIX

Old (2014 course code; the slides up to 2020 have the same lines, with the stiffness line commented out)

```
//std::vector<double>& resting_length =
      spring_spec->getRestingLengths();
...
double resting_length = spring_spec->getParameters()[0][1];
...
double spring_stiffness = spring_spec->getParameters()[0][0];
resting_length+=0.01*dt;
```

`resting_length` is a `double`: a copy. Adding to it changes nothing in IBAMR, and the spring never grows.

New (IB/06)

```
for (std::vector<double>& spring : spring_spec->getParameters())
{
    spring[1] += growth_rate * dt;
}
```

A reference to each spring's parameters, for every spring whose master is this point. In 0.19.0: `std::vector<std::vector<double>>& getParameters();`; the header says the values may be changed but the vectors must not be resized.

Why changing the parameters works at all: `IBStandardForceGen` keeps pointers to the specifications' numbers, so the next force evaluation sees the new rest length. The 2018 course code already had a reference, for the first spring only.

The 2014 course code; `examples/IB/06-growing-springs` (README, "History"); IBAMR 0.19.0's `IBSpringForceSpec.h`; the October 2026 code check.

BEAMS: GETMESHDEPENDENTCURVATURES() THEN AND NOW

Old (the March 2014 slides and the 2016-11 curvature example)

```
//There are three ways to get beam curvatures depending on the IBAMR
      verion.The newest way is uncommented.
//TinyVector<double, 2>& curv =
      beam_spec->getMeshDependentCurvatures()[0];
//IBTK::Vector<double, 2> curv =
      beam_spec->getMeshDependentCurvatures()[0];
Point& curv = beam_spec->getMeshDependentCurvatures()[0];
```

The middle form has no `&`: it would edit a copy, the trap of the previous slide.

New (IB/09; IB/10 the same)

```
for (IBTK::Vector& curv :
      beam_spec->getMeshDependentCurvatures())
```

Every beam of the node, by reference. In 0.19.0: `std::vector<IBTK::Vector>& getMeshDependentCurvatures();`; `Point&` still compiles, since `Point` and `Vector` are the same type. Chapter 10 explains the rest curvature.

The March 2014 module slides; the 2013 curvature decks and their 2016-11 examples; `examples/IB/09-curling-beam`, `IB/10-jellyfish-bell`; IBAMR 0.19.0's `IBBeamForceSpec.h`.

YOUR OWN SPRING LAW: THE 2013 AND THE 2014-AND-LATER SIGNATURES

Old (the 2013 deck, quoting IBAMR of the time)

```
inline void default_linear_spring_force(double F[NDIM],
    const double D[NDIM], const double& stf, const double& rst,
    const int& lag_mastr_idx, const int& lag_slave_idx)
...
registerSpringForceFunction(0, &default_spring_force,
      &default_spring_force_deriv);
```

A vector force from the displacement `D`, with stiffness and rest length as arguments.

2014 to now (unchanged to 0.19)

```
double vp_spring_force(double R, const double* params,
                       int lag_mastr_idx, int lag_slave_idx)
...
ib_force_fcn->registerSpringForceFunction(1, &vp_spring_force,
      &vp_spring_force_deriv);
```

The force's size from the length `R` and the spring's parameters (the numbers of the `.spring` line; by convention `params[0]` is the stiffness and `params[1]` the rest length, but a function may read them as it likes: `vp_spring_force` reads a force magnitude and a phase). In 0.19.0: `using SpringForceFcnPtr = double (*)(double R, const double* params, int lag_mastr_idx, int lag_slave_idx);`

The function index is the fifth column of a `.spring` line (chapter 6); chapter 9b builds one. Changed between the 2013 deck and the 2014 code (not versioned).

The 2013 deck on new spring functions; the lab's 2014 forced-tube-pump code; `examples/IB/11-heart-tube-pump`; IBAMR 0.19.0's `IBSpringForceFunctions.h`.

IBFE (1/7): `Mesh(NDIM)` becomes `Mesh(init.comm(), NDIM)`

Old (module 3803, 2014; the two-mesh decks)

```
Mesh mesh(NDIM);
Mesh mesh1(NDIM); Mesh mesh2(NDIM);
```

New (`Mesh(init.comm(), NDIM)` from module 2016-11 on; the `IBTKInit` lines are IBFE/01's, IBAMR 0.8.0 on)

```
IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
const LibMeshInit& init = ibtk_init.getLibMeshInit();
...
Mesh mesh(init.comm(), NDIM);
```

- The 2017 update guide: "There is an error that comes up between versions when defining the mesh. In main.C, change — `Mesh mesh(NDIM);` to `Mesh mesh(init.comm(), NDIM);`"
- Changed between module 3803 (February 2014) and module 2016-11, the course's two installs; the sources do not say which libMesh release made the communicator compulsory (chapter X2 lists libMesh 0.9.3 with module 3803 and 1.0.0 with module 2016-11).
- The `init` object now comes from `IBTKInit` (slide 10).

The 2014 IBFE examples and their README; the 2017 deck "How to update examples to new install"; `examples/IBFE/01-tethered-plate`.

IBFE (2/7): THE ARGUMENTS OF FORCE AND STRESS FUNCTIONS

Old (2014–2017)

```
target_force_function(
    VectorValue<double>& F,
    const TensorValue<double>& /*FF*/,
    const libMesh::Point& X,
    const libMesh::Point& s,
    Elem* const /*elem*/,
    const vector<NumericVector<double>*>& /*system_data*/,
    double time,
    void* /*ctx*/)
```

2018 on (the same 9 arguments in IBFE/01)

```
    const std::vector<const std::vector<double>*>& /*var_data*/,
    const std::vector<const std::vector<VectorValue<double>
      >*>& /*grad_var_data*/,
```

“In main.C, change the arguments of the following functions (force, stress)” (the 2018 Dogwood slides): one `system_data` argument became two, the values and the gradients of the systems the function asks for.

In 0.19.0, `libmesh_utilities.h`: `using VectorMeshFcnPtr = void (*)(libMesh::VectorValue<double>& F, const libMesh::TensorValue<double>& FF, const libMesh::Point& x, const libMesh::Point& X, libMesh::Elem* elem, const std::vector<const std::vector<double>*>& system_var_data, const std::vector<const std::vector<libMesh::VectorValue<double>>*>& system_grad_var_data, double data_time, void* ctx);` The old code calls the current position `X` and the reference position `s`; 0.19.0 calls them `x` and `X`: same order. Changed between module 2016-11 and module 2018-03; unchanged from 0.16 to 0.19.

IBFE (3/7): REGISTRATION, AND INITIALIZEFEEQUATIONSYSTEMS()

```
IBFEMethod::LagBodyForceFcnData target_force_data(target_force_function);  // 2014; the FcnData and register calls are unchanged
      since
IBFEMethod::PK1StressFcnData PK1_dev_stress_data(PK1_dev_stress_function);
IBFEMethod::PK1StressFcnData PK1_dil_stress_data(PK1_dil_stress_function);
PK1_dev_stress_data.quad_order =
      Utility::string_to_enum<libMeshEnums::Order>(input_db->getStringWithDefault("PK1_DEV_QUAD_ORDER","THIRD"));
...
ib_method_ops->registerPK1StressFunction(PK1_dev_stress_data);
ib_method_ops->registerPK1StressFunction(PK1_dil_stress_data);
ib_method_ops->registerLagBodyForceFunction(target_force_data);
EquationSystems* equation_systems = ib_method_ops->getFEDataManager()->getEquationSystems();
```

- 2018 (“Change the data_manager and equation_system lines”): `ib_method_ops->initializeFEEquationSystems();` and `FEDataManager* fe_data_manager = ib_method_ops->getFEDataManager();` come *before* the registration, then `fe_data_manager->getEquationSystems()`.
- Now (IBFE/01): register first, then `initializeFEEquationSystems()` and `getFEDataManager()->getEquationSystems()`; `initializeFEData()` later. Both orders work: registering only stores the data.
- In 0.19.0: `PK1StressFcnData(PK1StressFcnPtr fcn = nullptr, std::vector<IBTK::SystemData> system_data = {}, void* const ctx = nullptr, const libMesh::QuadratureType& quad_type = libMesh::INVALID_Q_RULE, const libMesh::Order& quad_order = libMesh::INVALID_ORDER), registerPK1StressFunction(const PK1StressFcnData& data, unsigned int part = 0)`.

The 2014 IBFE examples; the instructor’s 2018 porting slides; `examples/IBFE/01-tethered-plate`; IBAMR 0.19.0’s `FEMechanicsBase.h`.

IBFE (4/7): LIBMESH::ORDER, AND AUTOPTR BECOMES STD::UNIQUE_PTR

Old

```
Utility::string_to_enum<libMeshEnums::Order>(...)
AutoPtr<ExodusII_IO> exodus_io(
    uses_exodus ? new ExodusII_IO(mesh) : NULL);
AutoPtr<FEBase> fe(
    FEBase::build(dim, F_dof_map.variable_type(0)));
AutoPtr<QBase> qrule = QBase::build(QGAUSS, dim, FIFTH);
```

The 2022 UA fix

```
std::unique_ptr<ExodusII_IO> exodus_io(
    uses_exodus ? new ExodusII_IO(mesh) : NULL);
```

The 2022 slide: “Comment out the AutoPtr below and replace with std::unique_ptr . . . There may be other changes needed, but this is the only one I’ve encountered so far.”

New (IBFE/01)

```
Utility::string_to_enum<libMesh::Order>(
    input_db->getStringWithDefault(
        "PK1_DEV_QUAD_ORDER", "THIRD"))
std::unique_ptr<ExodusII_IO> exodus_io =
    uses_exodus ? std::make_unique<ExodusII_IO>(mesh) : nullptr;
```

- `libMeshEnums::Order` still compiles: libMesh 1.7.8 keeps `namespace libMeshEnums` for compatibility.
- `AutoPtr` does not: `error: 'AutoPtr' was not declared in this scope` (chapter 3). libMesh removed it (1.7.8 has none); the related IBAMR change (0.4.0): “IBAMR now requires that libMesh be built in such a way that libMesh::UniquePtr is the same class as std::unique_ptr”. The course code switched between the 2018-03 folders and the 2020 (0.6.0) and 2022 (UA) ones.

`examples/legacy` IBFE mains; the 2022 UA cluster deck and the 2024 Lecture 0; `examples/IBFE/01-tethered-plate`; IBAMR changelog 0.3.0 to 0.4.0; chapter 3.

IBFE (5/7): TWO OR MORE MESHES

Old (the 2018 UNC multi-plate folder; the 2020 lecture calls this the "ca. 2016 approach for 2 boundaries")

```
Mesh mesh1(init.comm(), NDIM);
Mesh mesh2(init.comm(), NDIM);
...
vector<Mesh*> meshes(2);
meshes[0] = &mesh1;
meshes[1] = &mesh2;
Pointer<IBFEMethod> ib_method_ops = new IBFEMethod(
    ...);   // meshes in place of &mesh
FEDataManager* fe_data_manager_1 =
    ib_method_ops->getFEDataManager(0);
ib_method_ops->registerInitialCoordinateMappingFunction(
    coordinate1_mapping_function, 0);
AutoPtr<ExodusII_IO> mesh1_exodus_io(
    uses_exodus ? new ExodusII_IO(mesh1) : NULL);
```

New (IBFE/08)

```
std::vector<MeshBase*> meshes = { &beam_mesh, &circle_mesh };
...
ib_method_ops->registerLagBodyForceFunction(
    beam_tether_force_data, 0);
ib_method_ops->registerPK1StressFunction(
    PK1_dev_stress_data, 0);
...   // PK1_dil for part 0, then both for part 1
...
// one std::make_unique<ExodusII_IO> per part
```

Changed in 0.4.0: "Classes in IBFE (CIBFEMethod, IBFEMethod, IBFESurfaceMethod) now consistently take pointers to libMesh::MeshBase instead of libMesh::Mesh." The 2020 lecture: "How they are handled has changed since my last examples (ca. 2016), but 2018 is also different from 2020."

Today's error with `vector<Mesh*>`: `no matching function for call to 'IBAMR::IBFEMethod::IBFEMethod(const char [11], ..., std::vector<libMesh::Mesh*>&, int)'` (chapter 3). Chapter 21 teaches several meshes.

The 2016 two-mesh deck and the 2020 IBFE lecture 5; `examples/legacy` multi-plate and plate-circle mains; `examples/IBFE/08-plate-hits-circle`; IBAMR changelog 0.3.0 to 0.4.0.

IBFE (6/7): SYSTEM NAMES

Old (the horseshoe crab, 2018–2024)

```
System& F_system = equation_systems->get_system<System>(
    IBFEMethod::FORCE_SYSTEM_NAME);
System& U_system = equation_systems->get_system<System>(
    IBFEMethod::VELOCITY_SYSTEM_NAME);
```

New

```
ib_method_ops->getForceSystemName()
ib_method_ops->getVelocitySystemName()
ib_method_ops->getCurrentCoordinatesSystemName()  // IBFE/10
```

Changed in 0.12.0: “Global static system names for IBFEMethod and related classes are deprecated”; still declared at 0.19, with a warning that names the accessor.

The static names were class-wide constants; the accessors return the names the object actually uses (“the current force system name”, in the deprecation message of `FEMechanicsBase.h`).

`examples/legacy` horseshoe-crab and two-wing mains; `examples/IBFE/01`, `IBFE/10`; IBAMR 0.19.0’s `FEMechanicsBase.h`; changelog 0.11.0 to 0.12.0.

IBFE (7/7): RESTARTS

Restarts, now in every IBFE program of this series (of the course programs, only the horseshoe crab and the two wings saved FE restart data, and the UA triangle appended to its ExodusII file):

```
restart_read_dirname =
    app_initializer->getRestartReadDirectory();
restart_restore_num =
    app_initializer->getRestartRestoreNumber();
... new IBFEMethod(..., &mesh, max_levels,
      /*register_for_restart*/ true,
      restart_read_dirname, restart_restore_num);
exodus_io->append(from_restart && IBTK_MPI::getRank() == 0);
ib_method_ops->writeFEDataToRestartFile(
    restart_dump_dirname, iteration_num);
```

Every IBFE example of this series lets only processor 0 reopen the ExodusII file on a restart, as above: “when every process reopens it, a restart on more than one process leaves a file that can no longer be read (tested with IBAMR 0.16.0 and 0.19.0)” (their READMEs). IBFE/01 and IBFE/10 reopened it on every process until October 2026, when a restart test of both on two processes showed the damage and the fix. Most course programs wrote no FE restart files.

`examples/legacy` horseshoe-crab and two-wing mains; `examples/IBFE/01`, `IBFE/10`; IBAMR 0.19.0’s `FEMechanicsBase.h`; changelog 0.11.0 to 0.12.0.

INPUT FILES (1/3): OBSOLETE KEYS, IF(), STRAY BACKSLASHES

item	old	now
obsolete keys	`use_IB_spread_operator = USE_IB_DELTA_FUNCTIONS` and `use_IB_interp_operator = USE_IB_DELTA_FUNCTIONS` (every course IBFE input, 2014–2024)	removed; SAMRAI databases ignore keys nobody reads, so they were harmless; IBAMR 0.19.0's library no longer reads them (one of its own example inputs still carries one)
muParser `if()`	`gcoef_function_0 = "if( X_1 <= -0.5*d,0.0,if( X_1 >= 0.5*d,0.0,4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*( (0.25*d*d) - X_1*X_1 ) ) )"` (the pulsatile channel)	`"(X_1 <= -0.5*d) ? 0.0 : ((X_1 >= 0.5*d) ? 0.0 : 4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*((0.25*d*d) - X_1*X_1))"` (IB/02)
stray backslashes	`}\\` and `\\` at the end of two UA IBFE inputs (the half plate and the rotating plate, committed 29 July 2024)	removed; they abort the parse

What IBAMR 0.19.0 says to the old `if()`: `muParserRobinBcCoefs::setDataOnPatch(): error: Unexpected token "if" found at position 0`. The ternary `a ? b : c` means the same and works in old and new muParser; IB/02's README tells the story.

`examples/legacy` inputs; `examples/IB/02-pulsatile-channel` (`input2d`, README); the October 2026 code check (C8, V2).

INPUT FILES (2/3): WINDOWS LINE ENDINGS, AND //

item	old	now
Windows line endings	a run log kept in a UA course folder (its first lines): `Warning in input2d at line 2 column 10 : Illegal character token in input`, `Error in input2d at line 2 column 10 : syntax error`, `P=00000:Program abort called in file ``../../../../SAMRAI-2.4.4/source/toolbox/inputdb/InputManager.C'' at line 172, P=00000:InputManager exiting...`	LF endings forced by the repository's `.gitattributes`: `examples/** text=auto eol=lf` (chapter 3, "The most common first error")
// at every line's end	the course inputs' workaround for Windows line endings: most IB inputs since 2014, and the UA IBFE inputs from July 2024, e.g. `MU = 0.01 //`	not needed with LF

SAMRAI's input reader treats the carriage return of a Windows line ending as an illegal character, and the run stops before it starts.

`examples/legacy` inputs and run log (lines 1–9 only); the repository's `.gitattributes`; the maintained READMEs ("History"); the October 2026 code check (V1, V4); chapter 3.

INPUT FILES (3/3): RUN LENGTH

item	old	now
run length	a fixed `DT` with a long `END_TIME`: 10^{-6} for 2 s, two million steps	a larger fixed `DT`, checked against the old one (IB/01: 10^{-4} in place of 10^{-6}, the band's shape the same to 4 digits); in the IB examples, output every `int(END_TIME/(50*DT))` steps, as in IBAMR's `ex1`; `END_TIME` cut where it was long (IB/03: 20 to 2 s)

The course inputs were written for the clusters' week-long queues; the maintained ones aim at minutes on four cores, with the time step checked against the old one before it was raised (each example's README, "History).

`examples/legacy` inputs and run log (lines 1–9 only); the repository's `.gitattributes`; the maintained READMEs ("History"); the October 2026 code check (V1, V4); chapter 3.

OUTPUT: U_X BECOMES U; NODAL FIELDS; CLOCKWISE TRIANGLES

- Velocity components. The course scripts and pictures use `U_x`, `U_y` (e.g. `AddPlot("Pseudocolor", "U_y", 1, 0)`). Changed in 0.19.0: “IBAMR no longer stores redundant component output: e.g., for INSStaggeredHierarchyIntegrator, the output velocity is stored by just U and not also U_x, U_y, and U_z.” Use `U[0]` and `magnitude(U)`, which work with 0.16 and 0.19 (chapter 26).
- Nodal fields. Since 0.12 the staggered solver writes `U`, `P` and `Omega` at the nodes (chapter 33 reads them).
- Triangles from gmsh. The course’s `triangle.msh` had all 1404 triangles clockwise; “IBAMR has checked for this since version 0.11.0 (2022)”, and IBFE/05 reorders them with `fix_orientation.py` (chapter 18).

IBAMR changelog 0.18.0 to 0.19.0; the October 2026 code check (a), C9; `examples/IBFE/05-gmsh-triangle` README.

JOB SCRIPTS: LSF, THEN SLURM, AND THE OLD SCRIPTS' SLIPS

old	problem	now
`bsub -q week -n 16 mpirun ./main3d input3d` (KillDevil, Kure)	LSF, the scheduler of Kure and KillDevil; Dogwood and Longleaf use Slurm ("killdevil used LSF", the 2018 Dogwood slides)	`sbatch job.slurm`
`--ntasks=16`, `--partition=debug_queue`, last line `./main2d input2d`	asks for 16 tasks and runs one (no `mpirun`); a UNC partition	`mpirun -np "$SLURM_NTASKS" ./build/main2d input2d`
`#SBATCG --output IBAMR2D.out`	a typo: the directive is ignored	`#SBATCH --output=%x-%j.out`
`--partition=528_queue`; 64 cores for 72 hours	a UNC partition; a large request	–
`--ntasks-per-node=94`, `--time=12:00:00`, `srun ./main2d input2d`	94 ranks for 12 hours for a small 2D run	4 tasks, minutes (IBFE/06)
`#!bin/bash`	the shebang lacks its slash	`#!/bin/bash`
no `module load` in any old script	–	`module load gnu13 openmpi5; source ~/autoibamr/configuration/enable.sh`

Today's `job.slurm` (IB/01): `#SBATCH --account=<group>`, `--partition=standard`, `--ntasks=4`, `--mem-per-cpu=4gb`, `--time=00:30:00`, the two lines above, `cd "$SLURM_SUBMIT_DIR"`, `mpirun`. Chapters 1 and 3 teach it; chapter X1 keeps the retired clusters.

The lab's LSF notes; `examples/legacy` scripts; the 2018 Dogwood slides; the 2022 UA deck; `examples/IB/01-rubber-band/job.slurm`; the October 2026 code check (V3).

IBAMR'S OWN PORTING SCRIPTS, AND WHAT THEY DO NOT COVER

- IBAMR ships `scripts/code_updates/`:
  - `misc_cpp11_updates.pl`: "replacing UniquePtr and AutoPtr with std::unique_ptr; replacing libMesh headers with <memory> system header; replacing boost objects with corresponding std:: objects";
  - `update_mpi_calls.pl`: "Perl script to replace SAMRAI_MPI calls with IBTK_MPI calls" (`perl update_mpi_calls.pl <directory>`).
- Neither touches the config headers, `IBTKInit`, `MeshBase*`, the function arguments, the build or the input files: those are the rest of this chapter.
- Other changes the sources show: `boost::array` became `std::array` in IBAMR's own interfaces (0.4.0; no course code used it); `LMarker` was removed in 0.13.0 (no course example used it); from 0.16.0 IBAMR needs C++17, Eigen 3.4 or newer and libMesh 1.7.0 or newer; the configure option `--with-libmesh-compile-mode` became `--with-libmesh-method` (December 2013, chapter X2).

IBAMR 0.19.0's `scripts/code_updates/README` and changelogs 0.3.0 to 0.4.0, 0.12.1 to 0.13.0, 0.15.0 to 0.16.0; the December 2013 notes.

THE ERAS ON ONE TABLE (1/3): IB PROGRAMS, 2009–2014

change	2009 code	February 2013	December 2013	module 3803 (2014)
node access	`getStashData`	`getNodeDataItem`	=	=
target point type	`std::vector< double>&`	`TinyVector< double,NDIM>&`	`Point&`	`Point&` (the two older lines commented out)
spring parameters	`getStashData`	`getRestingLengths()`	–	`getParameters()`, copied into a `double` (the copy bug)
config headers	`*_prefix_config.h`	–	commented out	IB: commented out; IBFE: `IBAMR_config.h`
start-up	PETSc, then SAMRAI	=	=	= (`PETSC_NULL`)
build	Makefile	=	=	=

"=": as in the column to its left (for start-up and build in 2013, assumed: the decks of that year do not show them). "–": the sources of that date do not show it. The columns are the dates of the decks and code, not IBAMR versions; slides 13–20 give each change with its code.

THE ERAS ON ONE TABLE (2/3): IB PROGRAMS, 2016 TO NOW

change	module 2016-11	Dogwood 2018-03	UA (0.7.1, `ib10`)	IBAMR 0.16, 0.19
node access	`getNodeDataItem`	=	=	= in a range-based `for`
target point type	`Point&`	=	=	`Point&` (`IBTK::Point`, an Eigen vector)
spring parameters	`getParameters()`, the copy bug	a reference, the first spring only	–	a loop over every spring, by reference
config headers	added to every program and header	=	=	removed
start-up	PETSc, then SAMRAI (`PETSC_NULL`)	= (`NULL`)	=	`IBTKInit` (0.8.0)
build	Makefile	=	=	CMake (0.8.0)

"=": as in the column to its left; "–": no UA folder does this. The UA column covers the folders built with `/contrib/<group>/ibamr/0.7.1` and `/groups/<group>/ib10` (slide 4).

The source inventory of this chapter (October 2026): the decks and code of each era.

THE ERAS ON ONE TABLE (3/3): IBFE PROGRAMS

change	module 3803 (2014)	module 2016-11	Dogwood 2018-03	UA (0.7.1, `ib10`)	IBAMR 0.16, 0.19
the mesh	`Mesh(NDIM)`	`Mesh( init.comm(), NDIM)`	=	=	=
function arguments	`system_data`	=	`var_data, grad_var_data`	=	=
set-up order	register, then `getFEData Manager()`	=	`initializeFE Equation Systems()` first	=	either order
ExodusII writer	`AutoPtr`	=	=	`std::unique_ptr` (2022)	`std:: make_unique`
several meshes	`vector<Mesh*>`	=	=	–	`vector< MeshBase*>` (0.4.0)
quadrature order	`libMeshEnums:: Order`	=	=	=	`libMesh::Order`

"=": as in the column to its left. "–": no UA example has two meshes. The three UNC folders built with Dogwood's `ibamr/0.6.0` module (2020: `7-*` and `9-*`) already use `std::unique_ptr`; the two `7-*` folders, which have two meshes, also `vector<MeshBase*>`.

The source inventory of this chapter (October 2026): the decks and code of each era.

THE MAP (1/3): THE IB COURSE FOLDERS AND THEIR MAINTAINED EXAMPLES

course folder (in `examples/legacy/` unless named otherwise)	maintained	what changed beyond the standard port
UNC `3-Example_2Drubber_band`; UA `3-IBAMR-Example_2Drubber_band`	IB/01	the UA copy (the UNC band is not closed: 643 springs, the spring `643 0` missing); `DT` $10^{-6} \rightarrow 10^{-4}$
UNC and UA `4-Example_2DPulsatileChannelFlow`	IB/02	muParser `if()` $\rightarrow$ `? :`; output intervals that scale (the course wrote 3 frames); unused `V = 0.1` removed
UNC and UA `4-Example_2DTetheredBeam`	IB/03	the no-op `update_target_point_positions` ($V = 0$) removed; a `V` input; `DT` $10^{-5} \rightarrow 10^{-4}$, `END_TIME` $20 \rightarrow 2$ s
UNC `5-Example_2DMovingCylinders`; UA `4-IBAMR-Example_2DMovingCylinders`	IB/04	comments say the motion is in y; new generators; the `#SBATCG` script dropped
UNC `5-Example_2DSprings`	IB/06	the rest length changed by reference for every spring; new generators
UNC `5-Example_2Dmoving_plates`	IB/07	headers trimmed; unused variables and the commented `update_springs()` removed
UNC and UA `4-Example_3DGlycocalyx`; the lab's 2014 `3DGlycocalyx`	IB/08	the 2014 options back as input keys; `SHEAR_START`; `V` really used; `END_TIME` $10 \rightarrow 0.2$
the 2013 beams-curvature and jellyfish-curvature examples (2016-11 copies)	IB/09, IB/10	rebuilt: indices from the structure's first index; inputs for the curl, the period and the contraction
the lab's 2014 `Forced-tube-pump`	IB/11	an `ACTUATION` input key instead of commented calls; one applied force for indices 1 and 2
UNC `8-AddingMass-Spirder`	IB/12	Boussinesq parts dropped; the beam loop fixed (no `169 170 171`); `output_data` added; 2 s

`examples/legacy`; the maintained examples' READMEs ("History"); the October 2026 code check (b). IB/13 and ConstraintIB/01–03 have no course folder.

THE MAP (2/3): THE IBFE COURSE FOLDERS

legacy folder	maintained	what changed beyond the standard port
UNC `1-IBFE-Example_TetherForceHalfPlate2D`, `...FullPlate2D-Rotate`; UA `...FullPlate2D`, `...HalfPlate2D`, `...FullPlate2D-Rotate`	IBFE/01	three folders → one program (`MOTION`); an exact rotation; the stray `\\` removed; mesh, stiffness and μ inputs; the UNC copies also `AutoPtr`
UNC `2-IBFE-Example_TetherForceFullTriangle2D`; UA `2-IBFE-UA-Example-2DTriangle`	IBFE/05	clockwise triangles fixed; `triangle.geo` added; processor-0 ExodusII reopen
UNC `4-IBFE-Example_{FiberStress2D, MovingFiberStress2D, AltMovingFiberStress2D}`; UA `4-IBFE_*FiberStress2D_UofA`	IBFE/06	three folders → `FIBER_MODE`; a 13 MB committed binary dropped; 94 ranks for 12 h → 4 cores
UNC `7-IBFE-Plate-Circle`	IBFE/08	already `MeshBase*`, `unique_ptr` and one ExodusII file per part; both parts restart; the files renamed `beam_output.ex2`, `circle_output.ex2`; unused mappings and a one-mesh `output_data` removed
UNC and UA `3-IBFE-Example_FlowTank-MidPlate`	IBFE/10	the empty `output_data` now computes the tether force; the 2017 options back (`TRANSLATE_X`, `TWO_MATERIALS`)
UNC `9-IBFE_TetherForceHalfPlate2D_advdiff`	IBFE/12	`ADV_DIFF_SOLVER_TYPE` at the top of `input2d`; a smooth periodic initial T

`examples/legacy`; the maintained examples' READMEs; the October 2026 code check (b).

THE MAP (3/3): FOUR PROGRAMS WITH NO SUCCESSOR, AND THE STANDARD PORT

legacy folder	maintained	what changed beyond the standard port
UNC `5-IBFE-Example_TetherForceMultiplePlate2D`	none	would need `AutoPtr` → `unique_ptr` twice and `vector<MeshBase*>`; its pattern is in IBFE/08
UNC and UA `3-IBFE-Example-HorseshoeCrab`	none	the UNC copy's five `AutoPtrs` (the UA copy uses `std::unique_ptr`), the static system names, and `U_integral` never initialized
UNC `7-IBFE-TwoWings`	none	static system names; its target *velocity* jumps at every half-stroke (the position does not; chapter 21)
one 2020 example folder	none	not part of the series

The standard port: a `CMakeLists.txt`; the config includes removed; `IBTKInit`; `IBTK::rel_equal_eps`, `IBTK_MPI`; LF inputs without `//`; a `job.slurm` for Puma; a larger `DT`, shorter runs and (IB) output intervals that scale with the run. IBFE adds `getLibMeshInit()`, `libMesh::Order`, `std::make_unique`, no `use_IB_*_operator`, FE restarts with ExodusII append.

`examples/legacy`; the maintained examples' READMEs; the October 2026 code check (b).

SUMMARY: THE ORDER TO FIX AN OLD PROGRAM

1. Date it by its Makefile path (slides 3–4); if a maintained example descends from it, start from that example (slides 37–39).
2. Otherwise start from a current example's `main` and move your parts in (slide 12): it brings `IBTKInit`, the right includes and the CMake build at once.
3. Fix what the compiler stops on: the config headers first (almost every course program stops there), then, in IBFE programs, `AutoPtr`, `vector<Mesh*>`, the function arguments and `Mesh(init.comm(), NDIM)`.
4. Check the silent ones: copies instead of references (springs, beams), `[first, second)` ranges, the run length.
5. Fix the inputs (line endings, `if()`, obsolete keys) and write a `job.slurm`.
6. Build against 0.16.0 and 0.19.0 if you can; run a few hundred steps; look at the output with `U`, not `U_x`.

> Chapter 3's porting checklist is this list in nine steps; the examples' READMEs ("History") say what was ported in each.